Which Evidence Is Worth Its Cost? A Pilot Study of Token-Cost-Aware Tool Use for LLM Alert Triage

AASHMA UPRETY, Southern Connecticut State University, USA

A large language model (LLM) that triages intrusion detection alerts can call tools, such as log queries, before it decides, but every tool output adds input tokens and therefore cost. This pilot measures the accuracy gain and the token cost of four evidence tools on alerts from the public AIT Log Data Sets. On 589 Suricata alert cases (25 attacks), Claude Haiku 5.5 without tools reaches 68.7% balanced accuracy; with the activity of the remote IP address as its only tool it reaches 97.0%, above Claude Opus 5.5 without tools (87.3%) at 1/18 of the token cost. On 2,235 Wazuh and Suricata alert cases from eight attacked networks, each network held out in turn, a policy that learns from labeled outcomes which tools to call for each alert signature raises Haiku from 80.9% to 93.0% and Opus from 88.4% to 97.0% at nearly the cost of no tools. A supervised random forest reaches 96.8% on these networks, but drops to 74.2% when trained on one network, where the routing policy keeps 92.3% (Haiku). These results show, on simulated data, that the choice of evidence matters more than the choice of model. They do not establish performance on real security operations center (SOC) traffic or on attacks unlike the scripted ones in the data sets.

Additional Key Words and Phrases: intrusion detection, alert triage, large language models, tool use, token cost

1 Introduction and Background

Intrusion detection systems such as Suricata [13] raise an alarm whenever a rule matches, and most alarms are false positives caused by legitimate activity [1]. Analysts who must check every alarm suffer from alert fatigue and miss attacks [7]. LLMs have been used to classify alerts from a fixed input [14, 15] and, as agents [17], to query logs before they decide [4, 16]. Eilertsen et al. [4] showed on the same data that models without tools miss the attacks that an agent with log access finds. Tsegai et al. [16] found that more tokens did not improve agent triage. Both used a fixed tool set and did not measure what each tool adds or costs.

Each tool output is text that the model must read, so evidence raises the cost of each of thousands of daily decisions. Cost-aware inference routes queries between cheap and expensive models [3, 12] or learns when to acquire information [5]; Lee et al. [11] computed expensive IDS features only when needed. This pilot applies the same question to LLM evidence: how much does each tool add to the triage decision, what does it cost in tokens, and can a low-cost model with the right evidence replace an expensive model?

2 Methods

2.1 Data and alert cases

One attack tool can trigger thousands of identical alarms, so we group them. Let alarms $e_1, e_2, \ldots$ with the same signature, source, and destination address be ordered by time $t(e_j)$. They belong to the same *alert case* as long as

$$t(e_{j+1}) - t(e_j) \le \Delta, \qquad \Delta = 10 \text{ min}. \tag{1}$$

An alert case a_i has a signature s_i, its addresses, a number of alarms, a time span $[t_i^s, t_i^e]$ from its first to its last alarm, and a label $y_i \in \{0, 1\}$, with $y_i = 1$ for attacks. In the rest of the paper, *alert* means alert case. In *Setting A*, the AIT Log Data Set V1.1 [10] provides six days of logs of four mail servers; on day five an attacker runs scans, password guessing, a webshell, and an Exim exploit against each server. Its 14,124 Suricata alarms form 589 cases, 25 of them malicious. In *Setting*

Author's Contact Information: Aashma Uprety, Southern Connecticut State University, New Haven, CT, USA.

C, AIT-LDS V2.0 [8] and the AIT Alert Data Set [9] provide Wazuh and Suricata alarms of eight simulated company networks, each attacked by a multi-step intrusion. We sample all malicious cases (capped at 20 per network and signature; 235 cases) and 250 benign cases per network (2,000), half with signatures that also occur in attacks. Labels come from the data sets' line labels; a case is malicious if a labeled line or the attacker's own address is involved during an attack phase.

2.2 Evidence tools

Four tools $\tau_1, \ldots, \tau_4$ return a text summary of the logs of the alerted host. Tool τ_k reads the interval $[t_i^s - w_k,\ t_i^e]$, where w_k is its look-back period, so it never reads data recorded after the alert. The *alert-history* tool reports how often the signature fires per day on this host. The *context* tool lists other alarms before and during the case. The *IP-activity* tool summarizes what the remote address did (requests, user agents, failed logins, DNS queries) and on which earlier days it was active. The *host* tool lists commands from the audit log (Setting A) or account activity (Setting C). Adding the tools one at a time gives the tool sets

$$S_0 = \emptyset, \qquad S_k = \{\tau_1, \ldots, \tau_k\}, \quad k = 1, \ldots, 4, \tag{2}$$

from no tools (S_0) to all tools (S_4); we also run each tool alone. Given alert a_i and the evidence of the tools in S, the LLM returns a verdict $\hat{y}_i(S) \in \{0, 1\}$ and a confidence $c_i \in [0, 1]$ that the alert is malicious.

2.3 Models, prompts, and cost

The system prompt sets the analyst role, describes the network, and states that most alerts are false positives. The user prompt lists the alert (signature, endpoints, number of alarms, time span, up to five example lines) and one text block per tool, and requests a JSON verdict, attack type, confidence, and reason. An unparsable answer counts as an escalation. We use Claude Haiku 5.5 (0.10/0.50 USD per million input/output tokens), Claude Opus 5.5 (4/20), Gemini 3.7 Flash (0.75/3.75, introductory), and Gemini 3.5 Flash-Lite (0.30/2.50) through Google Cloud Vertex AI [2, 6]. The token cost of triaging the alert set $\mathcal{A}$ with model $\mathcal{M}$ and tool set S is

$$C_{\mathcal{M}}(S) = \frac{1}{|\mathcal{A}|} \sum_{i \in \mathcal{A}} \left(\pi_{\mathcal{M}}^{\text{in}}\, n_i^{\text{in}}(S) + \pi_{\mathcal{M}}^{\text{out}}\, n_i^{\text{out}}(S) \right), \tag{3}$$

where $\pi_{\mathcal{M}}^{\text{in}}$ and $\pi_{\mathcal{M}}^{\text{out}}$ are the prices per input and output token, $n_i^{\text{in}}(S)$ counts the input tokens (instructions, alert, and evidence), and $n_i^{\text{out}}(S)$ the generated tokens, including reasoning tokens. We report $C_{\mathcal{M}}$ in USD per 1,000 alerts. It excludes tool compute, infrastructure, and analyst time. The value of ladder step k is its accuracy gain per unit of added cost,

$$\text{ROI}_k = \frac{\text{BA}(S_k) - \text{BA}(S_{k-1})}{C_{\mathcal{M}}(S_k) - C_{\mathcal{M}}(S_{k-1})}, \tag{4}$$

with BA as defined in Eq. (6).

2.4 Tool-selection policies and baselines

Besides the fixed tool sets, *P2 routing* looks up the signature s of a new alert and calls the tool set

$$\pi(s) = \arg\min_{S \in \mathcal{S}} \sum_{i \in \mathcal{A}_s} \left(\mathbf{1}\left[\hat{y}_i(S) \neq y_i\right] + \lambda\, \frac{C_{\mathcal{M},i}(S)}{\bar{C}_{\mathcal{M}}(S_0)} \right), \tag{5}$$

where $\mathcal{S}$ contains the ladder steps and the single tools, $\mathcal{A}_s$ are the labeled training alerts with signature s, $C_{\mathcal{M},i}(S)$ is the token cost of alert i (the summand of Eq. (3)), and $\bar{C}_{\mathcal{M}}(S_0)$ is the mean cost without tools. Thus λ is the price of one error in units of the cost of one alert without tools.

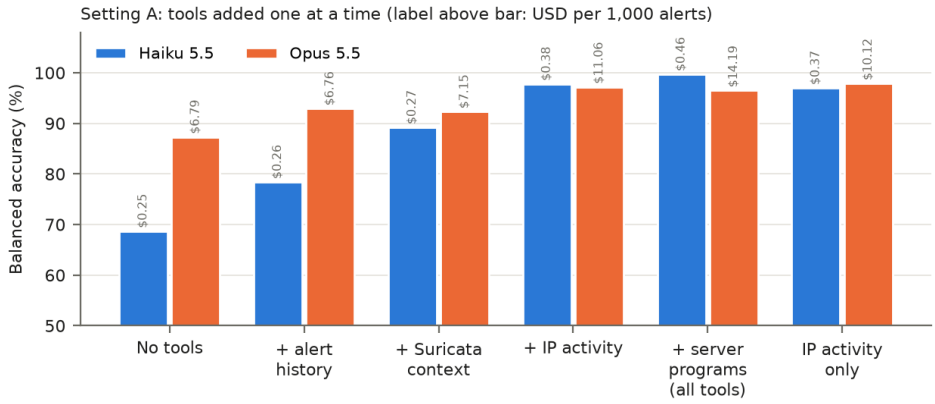


Fig. 1. Setting A (589 Suricata alert cases): balanced accuracy as tools are added, and with the IP-activity tool alone. The axis starts at 50%, the balanced accuracy of guessing. Labels give the token cost in USD per 1,000 alerts.

We fix $\lambda = 0.05$ on Setting A, before Setting C was collected. A signature without training alerts receives the IP-activity tool alone. Labels are needed only to learn the table π; the LLM triages without them. *Global routing* solves Eq. (5) once over all training alerts instead of per signature.

Confidence escalation first queries the model with S_0 and moves to the next ladder step while $c_i \in [\ell, h]$; the cost of an alert is the sum over all queried steps. We also compare with tool selection by the model itself, a Haiku-to-Opus cascade [3], and three baselines without an LLM: the IDS priority or Wazuh level, a rarity score in the spirit of NoDoze [7], and a random forest on alarm-stream features [14].

2.5 Evaluation

Attacks are rare, so we report the balanced accuracy, the mean of the detection rate (TPR) and the true-negative rate (TNR):

$$\mathrm{BA}(S) = \frac{1}{2}\left(\underbrace{\frac{\sum_i \mathbf{1}[\hat{y}_i(S) = 1,\ y_i = 1]}{\sum_i \mathbf{1}[y_i = 1]}}_{\mathrm{TPR}} + \underbrace{\frac{\sum_i \mathbf{1}[\hat{y}_i(S) = 0,\ y_i = 0]}{\sum_i \mathbf{1}[y_i = 0]}}_{\mathrm{TNR}} \right). \tag{6}$$

Guessing gives BA = 50% at any attack rate. Learned methods are trained on three hosts and tested on the fourth (Setting A), or trained on seven networks and tested on the eighth (Setting C), for every host or network h in turn: π is learned on $\mathcal{A} \setminus \mathcal{A}_h$ and applied to $\mathcal{A}_h$. API models were run three times and we report the mean. For a difference $\delta = \mathrm{BA}_X - \mathrm{BA}_Y$ between two methods, we draw $B = 5{,}000$ resamples of the alerts, with replacement and separately within attacks and benign alerts, evaluate both methods on the same resample b, and report the 95% confidence interval (CI) $[\delta^*_{(0.025)}, \delta^*_{(0.975)}]$ given by the 2.5th and 97.5th percentiles of the δ^*_b.

3 Results

3.1 Setting A: which tool pays off

Fig. 1 shows the tool ladder. Without tools, Haiku detects 9.3 of the 25 attacks and Opus 18.7, each with at most one false alarm among 564 benign cases. Every tool raises Haiku's detections (9.3, 14.3,

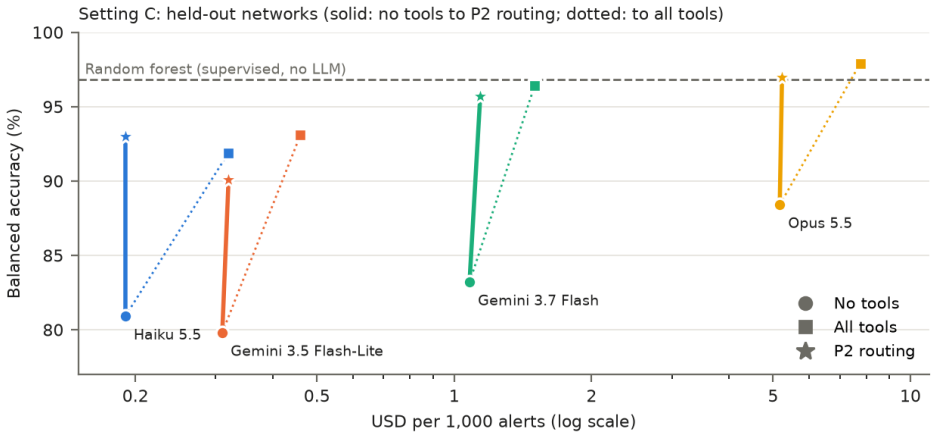


Fig. 2. Setting C (2,235 alert cases, eight networks, each held out in turn): balanced accuracy against token cost for no tools, all tools, and P2 routing. Mean of three runs.

19.7, 24.0, 25.0) while its false alarms stay at three or fewer, so all tools give 99.7% BA at 0.46 USD per 1,000 alerts. The IP-activity tool alone brings Haiku to 97.0% at 0.37 USD, 9.7 points above Opus without tools (CI +2.5 to +17.7) at 1/18 of its cost. For Opus, the full ladder adds 4.6 detections and doubles the cost. Not every tool helps: the host tool alone lowers Haiku to 66.4%, because routine server commands during a network attack lead the model to dismiss the alert.

Among the policies, P2 routing brings Haiku to 95.9% at 0.27 USD and calls the IP-activity or host tool for 2% of the alerts. Escalation on the model's confidence reaches only 80.0%, because Haiku gives its missed attacks confidences of 0.05–0.30, like benign alerts; when Haiku chooses its own tools, it answers 72% of the alerts without any. The random forest reaches 97.9%, not significantly different from Haiku with all tools.

3.2 Setting C: held-out networks

Fig. 2 shows that the pattern holds for four models on networks unseen in training. P2 routing raises Haiku from 80.9% to 93.0% at its cost without tools (0.19 USD), Gemini 3.7 Flash from 83.2% to 95.7%, and Opus from 88.4% to 97.0%. Gemini 3.5 Flash-Lite with all tools reaches 93.1% at 0.46 USD, above Opus without tools (CI +1.6 to +7.6) at 9% of its cost; for this model, P2 calls too few tools and stays at 90.1%. With all tools, Haiku detects 212 of 235 attacks but raises 132 false alarms, mostly on mail clients that log in many times per hour; P2 avoids them (1.7 false alarms). One tool set for all alerts, learned with the same objective, is 5.7–12.1 points less accurate than P2.

The random forest reaches 96.8%, above Haiku with P2 (CI +1.4 to +6.3) and on par with Opus, because the same scripted attack chain hits all eight networks. Trained on a single network (about 280 labeled alerts), the random forest drops to 74.2%, while P2 keeps 92.3% for Haiku and 97.5% for Opus.

3.3 Robustness checks

A reworded prompt changed BA by at most 1.6 points. Renaming addresses, domains, and hosts changed verdicts no more often than a repeated run did. For signatures removed from training, P2 reaches 92.1–95.8%; calling all tools for signatures without history bounds the worst case at the accuracy of all tools. In Setting C, stricter labels that count only labeled log lines raise every BA.

4 Future Work

The next step is an evaluation on alerts from a production SOC, with analyst time included in the cost. Further work will test attacks whose type is absent from training, update the routing table online as analysts label alerts, and compare evidence selection with fine-tuned small models.

5 Conclusion

In this pilot, a low-cost LLM (Haiku or Flash-Lite) with the right evidence was more accurate than an expensive LLM without evidence at 11 to 27 times lower token cost, and a per-signature routing table learned from a few hundred labeled alerts kept most of the gain at nearly the cost of no tools. The prompts, code, and result tables are available at https://github.com/aashmauprety/cotriage.

References

[1] Bushra A. Alahmadi, Louise Axon, and Ivan Martinovic. 2022. 99% False Positives: A Qualitative Study of SOC Analysts' Perspectives on Security Alarms. In *Proc. 31st USENIX Security Symposium*. Boston, MA, USA.

[2] Anthropic. 2026. Claude Pricing. https://platform.claude.com/docs/en/about-claude/pricing. Accessed Oct. 26, 2026.

[3] Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research* (2024).

[4] Even Eilertsen, Vasileios Mavroeidis, and Gudmund Grov. 2026. Towards Agentic Investigation of Security Alerts. arXiv:2604.25846.

[5] Yicheng Feng, Yan Zhang, Yan Cheng, and Wei Qi. 2026. Scores Are Not Decisions: Cost-Aware Stopping for Tool Acquisition in LLM Agents. arXiv:2607.27083.

[6] Google Cloud. 2026. Vertex AI Generative AI Pricing. https://cloud.google.com/vertex-ai/generative-ai/pricing. Accessed Oct. 26, 2026.

[7] Wajih Ul Hassan, Shengjian Guo, Ding Li, Zhengzhang Chen, Kangkook Jee, Zhichun Li, and Adam Bates. 2019. NoDoze: Combatting Threat Alert Fatigue with Automated Provenance Triage. In *Proc. Network and Distributed System Security Symposium (NDSS)*.

[8] Max Landauer, Florian Skopik, Maximilian Frank, Wolfgang Hotwagner, Markus Wurzenberger, and Andreas Rauber. 2023. Maintainable Log Datasets for Evaluation of Intrusion Detection Systems. *IEEE Transactions on Dependable and Secure Computing* 20, 4 (2023), 3466–3482. doi:10.1109/TDSC.2022.3201582

[9] Max Landauer, Florian Skopik, and Markus Wurzenberger. 2024. Introducing a New Alert Data Set for Multi-Step Attack Analysis. In *Proc. 17th Cyber Security Experimentation and Test Workshop (CSET)*. ACM, 41–53.

[10] Max Landauer, Florian Skopik, Markus Wurzenberger, Wolfgang Hotwagner, and Andreas Rauber. 2021. Have It Your Way: Generating Customized Log Datasets With a Model-Driven Simulation Testbed. *IEEE Transactions on Reliability* 70, 1 (2021), 402–415. doi:10.1109/TR.2020.3031317

[11] Wenke Lee, Wei Fan, Matthew Miller, Salvatore J. Stolfo, and Erez Zadok. 2002. Toward Cost-Sensitive Modeling for Intrusion Detection and Response. *Journal of Computer Security* 10, 1–2 (2002), 5–22.

[12] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2025. RouteLLM: Learning to Route LLMs from Preference Data. In *Proc. Int. Conf. on Learning Representations (ICLR)*.

[13] Open Information Security Foundation. [n. d.]. Suricata: Open Source Network Threat Detection Engine. https://suricata.io. Accessed Oct. 2026.

[14] M. Rieger, A. Shah, A. Alam, and M. J. Hossain. 2026. Possibilities and Limitations of Using Large Language Models (LLMs) for Alert Classification and Prioritisation in Security Operations Centers (SOCs). *Expert Systems with Applications* 331 (2026), 133194. doi:10.1016/j.eswa.2026.133194

[15] Alina Schärmer, Max Landauer, Florian Skopik, Markus Wurzenberger, and Marco Squarcina. 2026. Let the Alerts Speak: LLM-Based IDS Alert Interpretation for SOC Triage. In *Proc. ARES 2026 Workshops, LNCS*, Vol. 16901. Springer, 346–364. doi:10.1007/978-3-032-35579-9_19

[16] Saimon Amanuel Tsegai, Alex Kantchelian, Danfeng (Daphne) Yao, and Peng Gao. 2026. From Investigation Failures to Reliable SOC Agents: Understanding and Improving LLM-Based Alert Triage. arXiv:2610.10608.

[17] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proc. Int. Conf. on Learning Representations (ICLR)*.